KingCobra Language Tutorial for Beginners

Written for KingCobra 1.0.0. Every example in this tutorial was run on the real interpreter, and the output shown is what it printed.

This tutorial teaches the **KingCobra language itself**: how to write programs, store data, make decisions, repeat work, build your own types, and handle errors. It does not cover the standard library modules (`std.http`, `std.nn`, and so on) or the compiler tools such as `kcc` and `kasm`.

No programming experience is assumed. Read the chapters in order and type every example yourself. You learn a language by running it, not by reading it.

---

1. Your first program

1.1 Running code

KingCobra programs are plain text files ending in `.kc`. Create a file called `hello.kc`:

```
print("Hello, KingCobra!");
```

Run it from a terminal:

```
kingcobra hello.kc
```

Output:

```
Hello, KingCobra!
```

Three other ways to run code:

- `kingcobra -e 'print(1 + 2)'` runs a snippet directly from the command line.
- `kingcobra repl` starts the **REPL**, an interactive prompt where you type one line and see the result immediately. It is perfect for experiments.
- `kingcobra check file.kc` checks a file for mistakes without running it.

1.2 The shape of a program

A program is a list of **statements**. Most statements end with a semicolon `;`. You can write statements at the top level of the file, and you can also put your code inside a function called `main`, which runs automatically:

```
print("top-level code runs first");

fn main() {
    print("then main runs");
}
```

In this tutorial the examples use `fn main() { ... }`, which is the tidiest habit for real programs.

1.3 Comments

Comments are notes for humans. The computer ignores them.

```
// a one-line comment
/* a comment that
   spans several lines */
```

A comment starting with three slashes, `///`, placed directly above a function or struct, becomes its documentation. You can read it later with `help(name)`.

1.4 Printing

`print` shows values separated by spaces and ends the line. `println` does the same. You can give it many values at once:

```
print("Ada", 31, true);
```

```
Ada 31 true
```

---

2. Values and types

Every value has a **type**. The basic ones:

Type	Examples	Notes
`int`	`42`, `-7`, `1_000_000`, `0xFF`, `0b1010`	Exact 64-bit whole numbers. `_` is just a visual separator.
`float`	`3.14`, `1e9`, `2.5E-3`	Numbers with a fractional part.
`string`	`"hello"`	Text in double quotes.
`bool`	`true`, `false`	Yes or no.

Type	Examples	Notes
`null`	`null`	"No value".
`array`	`[1, 2, 3]`	An ordered list.
object	`{ name: "ada" }`	A set of named fields.

A character in single quotes, such as `'a'`, is actually an **int** holding the character's Unicode number (`'a'` is `97`).

Ask any value for its type with `typeof`:

```
print(typeof(1), typeof(1.5), typeof("s"), typeof(true), typeof(null),
typeof([1]));
```

```
int float string bool null array
```

2.1 Converting between types

```
print(int("42") + 1, str(12) + "!");
```

```
43 12!
```

`int(...)`, `float(...)` and `str(...)` convert a value to that type. Note that `print` shows a whole-valued float without the decimal point, so `float(3)` prints as `3`.

---

3. Variables

A **variable** is a named box that holds a value. KingCobra has three ways to declare one:

```
let name = "Ada";      // cannot be changed later
var age = 30;          // can be changed later
const PI = 3.14159;    // same as let, but says "this is a fixed constant"

age += 1;              // fine: age is a var
print(name, age, PI);
```

```
Ada 31 3.14159
```

If you try to change a `let`, KingCobra stops you and explains:

```
let y = 5;
y = 6;
```

```
error: cannot assign to 'y': it is immutable (declare it with `var`)
```

Rule of thumb: start with `let`. Switch to `var` only when the value really needs to change.

3.1 Optional type annotations

You may state a variable's type after a colon. This is optional, but it lets KingCobra catch mistakes before your program runs:

```
let x: int = 5;
let s: string = 1 + 2;   // error: declared string, but initializer is int
```

When you leave the annotation out, KingCobra works out the type from the value.

---

4. Numbers and operators

4.1 Arithmetic

```
print(7 / 2, 8 / 2, idiv(7, 2), 7 % 3);
```

```
3.5 4 3 1
```

- `/` gives an int when the division is exact (`8 / 2` is `4`) and a float otherwise (`7 / 2` is `3.5`).
- `idiv(a, b)` always rounds down (floor division).
- `%` is the remainder.
- Dividing by zero is an error, not a crash.
- For powers use the function `pow(2, 10)`, which gives `1024`.

> **Careful:** the symbol `^` is **not** "to the power of". It is bitwise XOR (see below), so `2 ^ 10` is `8`.

4.2 Comparison and logic

```
let x = 5;
print(x > 3 && x < 10, !true, x == 5 ? "five" : "other");
```

```
true false five
```

Operator	Meaning
`== !=`	equal, not equal (compares **contents** for arrays and structs)
`< <= > >=`	ordering
`&& || !`	and, or, not
`cond ? a : b`	pick `a` if `cond` is true, otherwise `b`
`a ?? b`	use `a`, but if it is `null` use `b`
`x in c`	is `x` an element of the array, a substring, or a key?
`+= -= *= /= %=`	update a variable in place (`x += 1`)
`++ --`	add or subtract one (`x++`)

4.3 Bitwise operators (a peek)

```
print(1 << 4, 6 & 3, 6 | 3, 6 ^ 3, ~0, 16 >> 2);
```

```
16 2 7 5 -1 4
```

These work on the individual bits of integers. You can ignore them until you need them.

4.4 Integers wrap around

Integers are exactly 64 bits. If an `int` addition overflows, it wraps around like in C instead of becoming a float.

---

5. Strings

5.1 Basics

```
let st = "Hello, World";
print("abc" + "def", len(st), st.upper());
```

```
abcdef 12 HELLO, WORLD
```

Escape sequences use a backslash: `\n` (new line), `\t` (tab), `\\` (a backslash), `\"` (a quote). A string may also span several lines.

5.2 f-strings: putting values inside text

Put an `f` in front of the quote and write any expression inside `{ }`:

```
let name = "Ada";
let age = 31;
print(f"{name} is {age} years old, next year {age + 1}");
```

```
Ada is 31 years old, next year 32
```

Use `{{` and `}}` for literal braces. A string **without** the `f` never interpolates, so text with braces (like JSON) is safe.

5.3 Handy string methods

```
let st = "Hello, World";
print(st.lower(), st.split(", "), st.contains("World"));
print(st.replace("World", "KC"), st.starts_with("He"), "  x ".trim());
print("ab".repeat(3), "7".pad_left(3, "0"), st[0..5]);
```

```
hello, world [Hello, World] true
Hello, KC true x
ababab 007 Hello
```

`st[0..5]` is a **slice**: characters from position 0 up to (but not including) 5. Positions start at **0**.

Looping over a string gives one-character strings:

```
fn reverse_string(s: string) -> string {
    var out = "";
    for (ch in s) { out = ch + out; }
    return out;
}
print(reverse_string("cobra"));   // arboc
```

---

6. Making decisions

6.1 if / else

The condition goes in parentheses and the body in curly braces:

```
let n = 15;
if (n % 15 == 0) {
    print("FizzBuzz");
} else if (n % 3 == 0) {
    print("Fizz");
} else {
    print(n);
}
```

```
FizzBuzz
```

6.2 match: choosing between many values

`match` is an expression: it produces a value.

```
fn describe(n: int) -> string {
    return match (n) {
        0 => "zero",
        1 | 2 | 3 => "few",
        _ => "many",
    };
}
print(describe(0), describe(2), describe(50));
```

```
zero few many
```

Read `1 | 2 | 3` as "1 or 2 or 3". The underscore `_` is the catch-all. If no arm matches and there is no `_`, you get an error rather than a silent `null`.

6.3 switch (the C-style choice)

If you know C, `switch` works the same way, including fall-through:

```
var n = 0;
switch (n) {
    case 0: print("zero");
    case 1: print("one (fell through)"); break;
    default: print("other");
}
```

```
zero
one (fell through)
```

Beginners should prefer `match`, which has no fall-through surprises.

---

7. Repeating work: loops

7.1 while

```
var i = 0;
while (i < 3) {
    print("i =", i);
    i += 1;
}
```

```
i = 0
i = 1
i = 2
```

7.2 for ... in

`0..3` is a **range**: 0, 1, 2. The end is *not* included.

```
for (k in 0..3) { print("k", k); }          // k 0, k 1, k 2
for (v in ["a", "b"]) { print(v); }          // a, b
for (idx, v in ["a", "b"]) { print(idx, v); } // 0 a, 1 b

let o = { name: "ada", n: 1 };
for (key, value in o) { print(key, value); }   // name ada, n 1
```

There is no `1..=3` form. To include 3, write `1..4`.

7.3 break and continue

`continue` skips to the next round. `break` leaves the loop.

```
for (i in 0..10) {
    if (i == 2) { continue; }
    if (i == 5) { break; }
    print(i);
}
```

```
0
1
3
4
```

7.4 Labeled loops

Give an outer loop a label to break out of, or continue, the outer loop from inside an inner one:

```
outer: for (i in 0..4) {
    for (j in 0..4) {
        if (j == 2) { continue outer; }
        if (i == 2) { break outer; }
        print("pair", i, j);
    }
}
```

```
pair 0 0
pair 0 1
pair 1 0
pair 1 1
```

7.5 C-style loops

`do { } while (cond);` and `for (var j = 0; j < 3; j++) { }` also exist, for people who like C.

---

8. Functions

A **function** is a named, reusable piece of code.

```
fn add(a: int, b: int) -> int {
    return a + b;
}
print(add(2, 3));   // 5
```

- `fn` starts the definition. Do not forget it: writing `add(a, b) { ... }` without `fn` is an error.
- `a: int` is a **parameter** with its type.
- `-> int` is the type of the value the function returns. A function that returns nothing omits the arrow, and then it may not `return` a value.
- Calling with the wrong number of arguments is an error.

8.1 Default and variadic parameters

```
fn greet(name: string, greeting: string = "Hello") -> string {
    return f"{greeting}, {name}!";
}
fn total(first: int, ...rest: int) -> int {
    var t = first;
    for (x in rest) { t += x; }
    return t;
}
print(greet("Ada"), greet("Bo", "Hi"), total(1, 2, 3, 4));
```

```
Hello, Ada! Hi, Bo! 10
```

A default is used when the argument is left out. `...rest` collects any extra arguments into an array, and it must be the last parameter.

8.2 Recursion

A function may call itself. Always give it a case where it stops:

```
fn fact(n: int) -> int {
    if (n <= 1) { return 1; }
    return n * fact(n - 1);
}
print(fact(10));   // 3628800
```

8.3 Functions are values

A function without a name is a **closure**. You can store it in a variable and pass it around:

```
let double = fn(x: int) -> int { return x * 2; };
print(double(21));   // 42
```

A closure **remembers** the variables around it:

```
fn make_counter() -> fn {
    var c = 0;
    fn inc() -> int { c += 1; return c; }
    return inc;
}
let counter = make_counter();
print(counter(), counter(), counter());   // 1 2 3
```

8.4 Higher-order functions

Arrays have methods that take a function:

```
let nums = [1, 2, 3, 4, 5];
print(nums.map(fn(x) -> int { return x * x; }));
print(nums.filter(fn(x) -> bool { return x % 2 == 0; }));
print(nums.reduce(fn(a, b) -> int { return a + b; }, 0));
```

```
[1, 4, 9, 16, 25]
[2, 4]
15
```

`map` transforms every element, `filter` keeps some, `reduce` folds everything into one value.

---

9. Arrays

```
var a = [10, 20, 30];
a.push(40);                  // add to the end
print(a[0], len(a));         // 10 4
a[0] = 99;                   // change an element
print(a.contains(20), a.index_of(30));   // true 2
print(a[1..3]);              // [20, 30]  (a slice: a copy)
print(sorted([3, 1, 2]));    // [1, 2, 3]
```

Other useful array methods: `pop`, `reverse`, `sum`, `min`, `max`, `first`, `last`, `insert(i, v)`, `remove(i)`, `join`, `sort`, `find`, `any`, `all`, `each`, `extend`, `concat`.

Indexes are strict: `a[5]` on a 1-element array is an error you can catch, and negative indexes are not allowed.

9.1 Destructuring

Unpack an array into separate variables in one line:

```
let [first, second, ...rest] = [5, 3, 8, 1, 9];
print(first, second, rest);   // 5 3 [8, 1, 9]
```

9.2 The big gotcha: arrays are shared, not copied

```
let arr = [1, 2, 3];
let b = arr;        // b is another name for the SAME array
b.push(4);
print(arr);         // [1, 2, 3, 4]

let c = arr.slice(0, 2);   // slice makes a real copy
c.push(9);
print(arr, c);      // [1, 2, 3, 4] [1, 2, 9]
```

Arrays, structs and objects are **reference types**. Assigning one to another variable, or passing it to a function, does not copy it. If you want a copy, use `slice`.

---

10. Objects, structs, enums and methods

10.1 Objects: quick and flexible

```
let o = { name: "ada", langs: ["c", "kc"] };
print(o.name, o["name"], o.keys(), o.has("name"), len(o));
o["new"] = 1;               // objects can gain new keys
let { name, langs } = o;     // destructuring by field name
```

Reading a missing key with `o["zzz"]` gives `null`.

10.2 Structs: a fixed shape

When the fields are known in advance, declare a **struct**. It catches typos for you:

```
struct Point { x: int, y: int }

let p = Point { x: 3, y: 4 };
p.y = 10;
print(p.x, p.y);      // 3 10
```

You must give exactly the declared fields. An unknown field name is an error with a hint: `struct 'Point' has no field 'xx' (did you mean 'x'?)`.

10.3 impl: giving a type its own methods

```
struct Point { x: int, y: int }

impl Point {
    fn len2(self) -> int { return self.x * self.x + self.y * self.y; }
    fn origin() -> Point { return Point { x: 0, y: 0 }; }
}

let p = Point { x: 3, y: 4 };
print(p.len2());          // 25
print(Point.origin());    // Point { x: 0, y: 0 }
```

A function whose first parameter is `self` is called **on a value** (`p.len2()`). A function without `self` is called **on the type** (`Point.origin()`). You can even add methods to built-in types:

```
impl string { fn shout(self) -> string { return self.upper() + "!"; } }
print("hey".shout());   // HEY!
```

10.4 enum: a named set of choices

```
enum Color { Red, Green = 5, Blue }
print(Color.Red, Color.Green, Color.Blue);   // 0 5 6

for (name, value in Color) { print(name, value); }
```

Numbering starts at 0 and continues upward unless you set a value. Enums work nicely with `match` .

10.5 type aliases

```
type Score = int;
let s: Score = 10;
```

Declare an alias before you use it.

---

11. When things go wrong: errors

Every runtime error in KingCobra can be **caught**:

```
fn parse_age(text: string) -> int {
    let n = int(text);
    if (n < 0) { throw error("age cannot be negative"); }
    return n;
}

try {
    print(parse_age("30"));
    print(parse_age("-4"));
} catch (e) {
    print("rejected:", e.message);
}
```

```
30
rejected: age cannot be negative
```

- `throw` raises an error. You can throw `error("text")` or any value at all.
- Inside `catch (e)`, a runtime error such as `a[5]` gives an object with `e.message`, `e.line` and so on. A thrown value arrives unchanged.
- If nothing catches an error, the program prints the message with the source line and a caret, then exits with status 65.

11.1 defer: cleanup that always happens

`defer` schedules a statement to run when the current block ends, even if an error interrupts it:

```
for (i in 0..3) {
    defer print("deferred", i);
    print("body", i);
}
```

```
body 0
deferred 0
body 1
deferred 1
body 2
deferred 2
```

If you register several defers, they run in reverse order, last one first.

---

12. Splitting a program into files

Put reusable code in its own file, for example `mods/mathutils.kc`:

```
fn square(x: int) -> int { return x * x; }
let VERSION = "1.0";
```

Import it with a name of your choice:

```
import "mods/mathutils.kc" as mu;

fn main() {
    print(mu.square(7), mu.VERSION);   // 49 1.0
}
```

The path is looked up relative to the file doing the importing. A module only sees the built-in functions, not your other variables. Built-in library modules use a dotted name instead, for example `import std.system;`, and are called like `system.platform()`.

---

13. Memory and pointers: a gentle introduction

KingCobra is a systems language, so it lets you work with memory directly, but safely.

- **There is no garbage collector, by design.** Ordinary values (numbers, strings, arrays) are handled for you. When you request raw memory yourself you must give it back with `free`.
- `defer free(x);` is the standard way to make sure that happens.
- Mistakes such as using freed memory, freeing twice, or reading outside a block are **errors you see**, not mysterious crashes.

```
var x = 42;
let p = &x;      // p points at x
print(*p);       // 42   (read through the pointer)
*p = 7;          // write through the pointer
print(x);        // 7
```

You can only write through a pointer to a `var`. Pointing at a `let` and assigning gives an error. For raw memory, `buffer(8)` and `alloc(16)` create blocks, and `free(...)` releases them. Run `kingcobra --leaks file.kc` to see which blocks you forgot to free. Everything about pointers, casts and struct layout is explained in `docs/POINTERS.md` and `docs/MEMORY.md` once you are comfortable with this tutorial.

---

14. comptime: running code before the program starts

A `comptime` block runs once and its results become ordinary values:

```
comptime {
    fn fib(n: int) -> int { if (n < 2) { return n; } return fib(n - 1) +
fib(n - 2); }
    let FIB10 = fib(10);
}
fn main() { print(FIB10); }   // 55
```

Inside `comptime` you may not do things with side effects, such as writing files or using the network. KingCobra refuses and tells you which call was the problem.

---

15. Checking and testing your code

15.1 The built-in type checker

Before a program runs, KingCobra checks it for mistakes such as the wrong type or an undefined name:

```
fn add(a: int, b: int) -> int { return a + b; }
fn main() {
    let s: string = add(1, 2);
}
```

```
error: 's': declared string, but initializer is int
```

Error messages show the exact line with a caret under the problem, and often suggest a fix.

15.2 assert and the test runner

Write tests as functions whose names start with `test_`:

```
fn add(a: int, b: int) -> int { return a + b; }

fn test_add() { assert_eq(add(2, 3), 5); }
fn test_fail() { assert_eq(add(2, 2), 5, "math is broken"); }
```

Save it as `t_test.kc` and run `kingcobra test t_test.kc`:

```
t_test.kc
  ok   test_add
  FAIL test_fail
       assertion failed: math is broken: expected 5, got 4  (t_test.kc:3)

1 passed, 1 failed
```

`assert(cond, "message")` is the simpler form. `kingcobra new myapp` creates a ready-made project folder with a program and a test.

15.3 Getting help

`help("map")`, `help("string.split")`, `help("keywords")` or `kingcobra help map` explain built-ins right from the terminal. Mistype a name and it suggests the closest one.

---

16. Putting it together: a grade book

This program uses structs, methods, arrays, loops, f-strings and `reduce`:

```
struct Student { name: string, scores: array }

impl Student {
    fn average(self) -> float {
        if (len(self.scores) == 0) { return 0; }
        return self.scores.sum() / len(self.scores);
    }
    fn grade(self) -> string {
        let avg = self.average();
        if (avg >= 90) { return "A"; }
        else if (avg >= 80) { return "B"; }
        else if (avg >= 70) { return "C"; }
        return "F";
    }
}

fn main() {
    let students = [
        Student { name: "Ada", scores: [95, 88, 92] },
        Student { name: "Bo", scores: [72, 65, 80] },
        Student { name: "Cy", scores: [] },
    ];
    for (s in students) {
        print(f"{s.name.pad_right(5)} avg={s.average()} grade={s.grade()}");
    }
```

```
    let best = students.reduce(fn(a, b) -> Student {
        return a.average() >= b.average() ? a : b;
    });
    print("Top student:", best.name);
}
```

Output:

```
Ada   avg=91.6667 grade=A
Bo    avg=72.3333 grade=C
Cy    avg=0 grade=F
Top student: Ada
```

Read it slowly and make sure you can explain every line. Then change it: add a fourth student, add a grade "D", or print the class average.

---

17. Common beginner mistakes

Mistake	What happens	Fix
Forgetting `fn` before a function name	Syntax error with a "use `fn`" hint	Write `fn name(...) { ... }`
Changing a `let`	"it is immutable" error	Declare it with `var`
Writing `2 ^ 10` for a power	You get `8` (bitwise XOR)	Use `pow(2, 10)`
Expecting a copy from `let b = arr;`	Both names change together	Use `arr.slice(0, len(arr))` for a copy
`for (i in 1..=3)`	Syntax error	Ranges end before the last number: `1..4`
Missing `;` after a statement	"expected ;" error	Add the semicolon
Writing to a pointer to a `let`	"immutable variable" error	Declare the target with `var`
Printing an array in the same call that changes it	Output shows the changed array	Change first, print on the next line
Comparing a one-character string to a character literal (`ch == 'a'`)	The types differ, so it is false	Compare with `"a"`

---

18. Practice exercises

Try each one yourself first. Solutions follow.

1. Print the numbers 1 to 20, but write `Fizz` for multiples of 3, `Buzz` for multiples of 5 and `FizzBuzz` for both.
2. Write `is_prime(n: int) -> bool` and use it to collect all primes below 30 in an array.
3. Write a function that counts how many times the letter `a` appears in a word.
4. Create a `Rectangle` struct with `w` and `h`, and an `impl` with `area` and `perimeter` methods.
5. Write `safe_divide(a, b)` that throws an error when `b` is 0, and call it inside `try` / `catch`.

Solutions

```
// 2. primes
fn is_prime(n: int) -> bool {
    if (n < 2) { return false; }
    for (d in 2..n) {
        if (d * d > n) { break; }
        if (n % d == 0) { return false; }
    }
    return true;
}
fn main() {
    var primes = [];
    for (n in 0..30) { if (is_prime(n)) { primes.push(n); } }
    print(primes);   // [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    // 3. counting letters
    var count = 0;
    for (ch in "banana") { if (ch == "a") { count += 1; } }
    print(count);    // 3
}
```

For exercise 1, start from `for (i in 1..21)` and test `i % 15 == 0` first, as in chapter 6. For exercise 4, copy the `Point` example from chapter 10 and change the fields. For exercise 5, follow the `parse_age` example in chapter 11.

---

19. The preprocessor

KingCobra is a systems language, so it also speaks the languages systems programming grew out of: C and B. The next three chapters cover these as part of the core language. They are optional. Everything you learned so far (typed `fn`, `for (i in 0..n)`, `match`) works unchanged, and you can mix styles in one program.

The **preprocessor** is a text pass that runs *before* KingCobra reads your code. Lines that start with `#` are instructions to it. It rewrites the text, and then the normal language takes over. Line numbers are preserved, so error messages still point at the right line.

19.1 Constants and macros with #define

```
#define N 10                    // every N becomes 10
#define SQ(x) ((x) * (x))       // a macro with a parameter
```

A macro is pure text replacement. That is why `SQ` wraps `x` and the whole result in extra parentheses. Without them, `SQ(1 + 2)` would expand to `1 + 2 * 1 + 2`, which is 5, not 9. `#undef N` removes a definition.

Macros are **not** expanded inside strings, character literals or comments.

19.2 Choosing code at build time: #if, #ifdef, #ifndef

```
#ifndef WIDTH            // only if WIDTH is not already defined
#define WIDTH 8
#endif

#if WIDTH > 4
fn mode() -> string { return "wide"; }
#else
fn mode() -> string { return "narrow"; }
#endif
```

`#if` accepts integers, arithmetic and comparison operators, `&&`, `||`, `!` and `defined(NAME)`. A name that is not a macro counts as `0`.

You can define macros from the command line with `-D`. Here is a complete program to try. Save it as `p1.kc`:

```
#define N 10
#define SQ(x) ((x) * (x))
#define STR(x) #x
#define CAT(a, b) a##b
#define LOG(f, ...) printf(f, __VA_ARGS__)
```

```
#ifndef WIDTH
#define WIDTH 8
#endif
#if WIDTH > 4
fn mode() -> string { return "wide"; }
#else
fn mode() -> string { return "narrow"; }
#endif
fn main() {
    print(N, SQ(1 + 2), STR(hello));
    var CAT(my, count) = 5;
    print(mycount, mode(), WIDTH);
    LOG("%s has %d\n", "x", 3);
    print(__LINE__, __FILE__);
#ifdef DEBUG
    print("debug on");
#endif
}
```

Run `kingcobra p1.kc`:

```
10 9 hello
5 wide 8
x has 3
19 p1.kc
```

Now run `kingcobra -DDEBUG -DWIDTH=2 p1.kc`. The second line becomes `5 narrow 2`, and a final line `debug on` appears.

19.3 Splitting code with #include

```
#include "inc/util.kc"
#include "inc/util.kc"      // the second one does nothing
```

where `inc/util.kc` contains:

```
#pragma once
fn helper() -> int { return 7; }
```

The included text is pasted in place. `#pragma once` at the top of a file makes any repeat include a no-op, so `helper` is defined only once and `print(helper())` prints `7`. The path is searched next to the including file first, then in the current directory. The form `#include <name>` searches the folders listed in the `KC_INCLUDE` environment variable and `./include`.

`#include` pastes text, while `import "file.kc" as name;` (chapter 12) loads a separate module with its own scope. Use `import` for most things.

19.4 Stringizing, pasting and variadic macros

- `#x` turns a macro argument into a string: `STR(hello)` becomes `"hello"`.
- `a##b` glues two pieces into one name: `CAT(my, count)` becomes `mycount`.
- `...` and `__VA_ARGS__` pass any number of arguments through, as `LOG` does above.

19.5 Predefined names and seeing the result

`__FILE__`, `__LINE__`, `__COUNTER__`, `__KC__` and `__KC_VERSION__` are always defined, plus one of `__LINUX__`, `__WINDOWS__` or `__APPLE__`. Use them for platform-specific code:

```
#ifdef __WINDOWS__
    // Windows-only code
#endif
```

If a macro does something unexpected, run `kingcobra preprocess file.kc` to print the program as the language sees it after expansion. Other directives are `#error message` and `#warning message`.

---

20. C-style programming

These features will feel natural if you know C, and they are useful to anyone who wants shorter loops or tight control of flow.

20.1 The C for loop, ++ and --, and the comma

```
for (var i = 1, sq = 1; i <= 3; i++, sq = i * i) {
    printf("%d %d\n", i, sq);
}
```

```
1 1
2 4
3 9
```

The header has three parts: initialize (`var i = 1, sq = 1`), test (`i <= 3`), and step (`i++, sq = i * i`). Commas let you declare or step several variables. Every part is optional, so `for (;;) { ... }` loops forever until a `break`. When the header reads `name in something`, you get the ordinary `for ... in` loop from chapter 7.

`x++` and `x--` add or subtract one and work on any place you can assign to, such as `a[i]++`. The postfix form yields the old value and the prefix form (`++x`) the new one. A body can be a single statement without braces: `while (n > 0) n--;`.

`do { ... } while (cond);` runs the body first and tests afterwards, so it always runs at least once:

```
var n = 6, steps = 0;
do { n = n % 2 == 0 ? n / 2 : 3 * n + 1; steps++; } while (n != 1);
printf("steps=%d\n", steps);   // steps=8
```

20.2 switch with fall-through

```
fn describe(code: int) -> string {
    switch (code) {
        case 0: return "zero";
        case 1, 2, 3: return "small";
        default: return "large";
    }
    return "?";
}
print(describe(2), describe(9));   // small large
```

`case 1, 2, 3:` matches several values. Remember the C rule from chapter 6: execution **falls through** into the next case until it meets `break`, `return` or the end of the switch. `default` may stand anywhere and is taken when nothing else matches.

20.3 goto and labels

```
var k = 0;
again:
k++;
if (k < 3) goto again;
print("k", k);   // k 3
```

`goto` jumps to a label in the same function, either in the same block or an enclosing one, so it can leave nested loops for cleanup or retry code. It cannot jump *into* a nested block. For ordinary repetition a loop is clearer.

20.4 static variables

```
fn next_id() -> int {
    static counter = 100;    // set up once, remembered between calls
    return counter++;
}
print(next_id(), next_id());   // 100 101
```

20.5 Repeated arrays and more declarations

```
var z = [0; 4];                  // four zeros
z[1]++; ++z[2];
print(z);                        // [0, 1, 1, 0]
var grid = [[0; 2]; 2];          // two separate rows, not one shared row
grid[0][0] = 1;
print(grid);                     // [[1, 0], [0, 0]]
```

You can also declare several variables at once (`let a = 1, b = 2;`), write `typedef int myint;` as another spelling of `type myint = int;` , and join adjacent string literals (`"ab" "cd"`).

A `union` stores all its fields at the same place in memory. Because of that you use it through a pointer to memory you allocate:

```
union Bits { as_int: u32, as_float: f32 }
let b = alloc(sizeof(Bits)) as *Bits;
b->as_float = 1.5;
printf("0x%08X\n", b->as_int);   // 0x3FC00000: the bits of 1.5 as an integer
free(b);
```

20.6 printf

```
printf("%5.2f|%-5d|%05d|%x|%c|%s|%b\n", 3.14159, 42, 42, 255, 65, "str", 5);
```

```
 3.14|42   |00042|ff|A|str|101
```

Piece	Meaning
`%d` `%i` `%u`	whole number
`%x` `%X` `%o` `%b`	hexadecimal, octal, binary
`%f` `%e` `%g`	floating point

Piece	Meaning
`%c`	character, given as a code number
`%s`	a string, or **any** value
`%5d` `%-5d` `%05d` `%.2f`	width, left-align, zero padding, precision
`%%`	a literal percent sign

Too few arguments or an unknown conversion is an error, never silent garbage. Related functions: `sprintf`, `format`, `eprintf` (standard error), `puts`, `putchar`.

20.7 The C standard library, KingCobra style

```
print(strlen("hello"), toupper("ab"), isdigit('7'), strstr("hello", "ll"),
atoi("12"), floor(2.7), M_PI);
```

```
5 AB true 2 12 2 3.14159
```

Strings are values, so functions that C changes in place return a new string, and "pointer into the string" results are **indices** (`-1` means not found). Available groups:

Group	Examples
strings	`strlen strcmp strchr strstr strcat strdup atoi atof itoa`
characters	`isdigit isalpha isspace isupper toupper tolower`
math	`floor ceil round fabs pow fmod sin cos tan exp log sqrt`
process	`exit abort rand srand time`
constants	`NULL EOF INT_MAX INT_MIN EXIT_SUCCESS EXIT_FAILURE M_PI M_E`

Sorting and searching take a comparison function that returns a negative number, zero or a positive number:

```
var a = [9, 4, 7, 1];
qsort(a, fn(x, y) { return x - y; });          // sorts in place, stably
printf("%s idx=%d\n", a, bsearch(a, 7, fn(k, e) { return k - e; }));
// [1, 4, 7, 9] idx=2
```

20.8 main with arguments and an exit status

```
fn main(argc, argv) -> int {
    print(argc, argv[0] != null);
    return 3;
}
```

Run it as `kingcobra m1.kc x y`. It prints `3 true` (the script counts as the first argument), and the process exits with status 3. A plain `fn main() { }` still works, and `exit(code)` ends the program immediately.

---

21. B-style programming

B was Ken Thompson's language from which C grew. It has no types at all, only machine words. KingCobra accepts B-style code, which makes it easy to read or port old programs.

```
extrn putchar, printn;          // these names come from the runtime

gcd(a, b) {                     // no `fn`, no parameter types
    auto t;                     // `auto` declares a variable
    while (b != 0) { t = b; b = a % b; a = t; }
    return a;
}

main() {
    auto v[3], i, s;            // v is a vector with indices 0..3 (four
words)
    for (i = 0; i <= 3; i++) v[i] = i * i;
    printn(v[3], 10); putchar(10);          // print 9 in base 10, then a
newline
    printn(gcd(1071, 462), 10); putchar(10);
    s = strdup("hello");        // string literals are read-only, so copy
first
    lchar(s, 0, 'H');           // store a byte: s[0] = 'H'
    printf("%s %c\n", s, char(s, 1));   // char(s, 1) reads the byte at index
1
    printn(255, 16); putchar(10);
    putnumb(42); putchar(10);
}
```

Output:

```
9
21
Hello e
ff
42
```

The B rules in short:

- A function is written `name(params) { ... }` with no `fn`. A B function definition is recognized because a call is never followed by `{`.
- `auto` declares variables and vectors, and `extrn` names things that come from outside. It checks that they exist when the statement runs.
- `auto v[3]` makes **four** words, indices 0 to 3, exactly as in B.
- `char(s, i)` reads and `lchar(s, i, c)` writes one character.
- `printn(n, base)` prints a number in any base, `putnumb(n)` in decimal, `putchar(c)` prints one character.
- Labels, `goto`, `switch`, `?:` and `printf` are available too, so B code can use the C features of the previous chapter.

B's old `=+` and `=-` assignment operators and its word-addressed strings are not supported, because they clash with syntax that already has a meaning in KingCobra.

21.1 Mixing the styles

All three styles can live in the same file and call each other. A B-style `main()` can use `printf`, a typed `fn` can use `goto`, and a macro can wrap any of them. For new code that you will keep, the typed KingCobra style from chapters 1 to 18 gives you the most help from the type checker. Reach for the C and B forms when you are porting code, want C's compact loops, or just like them.

21.2 Exercises for chapters 19 to 21

1. Write a macro `MAX(a, b)` and use it to print the larger of two numbers. Remember the extra parentheses.
2. Use `#ifdef DEBUG` to print a message only when the program runs with `-DDEBUG`.
3. Use a C-style `for` loop with two variables (`i` counting up, `j` counting down) that prints pairs until they meet.
4. Write a `switch` that returns `"weekend"` for 6 and 7 and `"weekday"` for 1 to 5, using `case 1, 2, 3, 4, 5:`.
5. Write a B-style function `power(base, n)` using `auto` and a `while` loop, and print `power(2, 10)` with `printn`.

Compare your answers with the matching examples above, and use `kingcobra preprocess` to check what your macros expand to.

---

22. Calling C: the FFI

FFI stands for **foreign function interface**. It lets a KingCobra program call functions that live in compiled C libraries, such as the C standard library, the math library, or a library you wrote yourself. This is how KingCobra reaches operating system features and existing C code without anyone having to rewrite them.

There are two ways in. A declared `extern fn` gives you full control over the types. A native `import` is shorter but guesses the types. We will meet both.

22.1 Declaring a C function with extern fn

The form is:

```
extern fn name(type, ...) -> return_type from "library";
```

The `name` must be the exact name of the C function. The types tell KingCobra how to pass the arguments and read the result: integers are passed as machine words, and `float` is passed as a C `double`. Here is a complete program that calls the C library and the math library on Linux:

```
extern fn abs(x: int) -> int from "libc.so.6";
extern fn sqrt(x: float) -> float from "libm.so.6";
extern fn pow(x: float, y: float) -> float from "libm.so.6";
extern fn getpid() -> int from "libc.so.6";
extern fn toupper(c: int) -> int from "libc.so.6";

fn main() {
    print(abs(-42), sqrt(2.0), pow(2.0, 10.0), getpid() > 0,
chr(toupper(97)));
}
```

```
42 1.41421 1024 true A
```

The library file names depend on the operating system. The ones above are for Linux. Windows uses `.dll` files and macOS uses `.dylib` files, so check which names your system uses.

If the library cannot be found, KingCobra tells you exactly what went wrong:

```
error: extern 'abs': could not load library 'libnotthere.so': libnotthere.so:
cannot open shared object file: No such file or directory
```

22.2 Pointers and unsafe

Some C functions hand back memory addresses or expect one. Such an address is a **raw pointer**: KingCobra cannot check its bounds or lifetime, because C does not track them. So the rules are strict:

- Passing a pointer or buffer to C, and reading or writing through a raw pointer that came back from C, must happen inside an `unsafe { ... }` block.
- You are responsible for giving back memory that C allocated, by calling C's `free`.

```
extern fn malloc(n: int) -> *u8 from "libc.so.6";
extern fn memset(p: *u8, v: int, n: int) -> *u8 from "libc.so.6";
extern fn free(p: *u8) from "libc.so.6";
extern fn strlen(s: *u8) -> int from "libc.so.6";

fn main() {
    unsafe {
        let c = malloc(16);        // 16 raw bytes from C
        memset(c, 0x41, 15);       // fill 15 of them with 'A' (0x41)
        c[15] = 0;                 // the C string terminator
        print(c.read_cstr(), strlen(c));
        free(c);
    }
}
```

```
AAAAAAAAAAAAAAA 15
```

`*u8` means "pointer to unsigned 8-bit bytes". `c.read_cstr()` reads the bytes up to the zero terminator as a KingCobra string. The `unsafe` keyword is a signal to you: everything inside it needs extra care.

22.3 Importing a native library

The second way needs no declarations at all. First write a small C file, `mathutil.c`:

```
long square(long x) { return x * x; }
long cube(long x) { return x * x * x; }
double half(double x) { return x / 2.0; }
```

Compile it once from a terminal. The `-fPIC` flag is required for object files:

```
cc -fPIC -c mathutil.c -o mathutil.o
cc -shared mathutil.o -o libmu.so
```

Now import either file. A `.o` is linked automatically (this needs a C compiler on your `PATH`), and `.so`, `.dll` and `.dylib` files load as they are:

```
import "mathutil.o" as mu;
import "libmu.so" as lib;

fn main() {
    print(mu.square(6), mu.cube(4), mu.half(7.0), lib.square(9));
}
```

```
36 64 3.5 81
```

Because there is no declaration, KingCobra **guesses** the types from the arguments you pass. If all arguments are integers, everything is treated as 64-bit integers, including the result. If the first argument is a float, everything is treated as a `double`. That is why `mu.half(7.0)` works but `mu.half(7)` would pass an integer to a function expecting a `double`.

22.4 Which one to use

Situation	Use
A function with mixed types (ints in, float out), or any pointer	`extern fn` with exact types
Your own simple C helpers with all-integer or all-float arguments	`import "lib" as name`
Code that should run at compile time	Neither: `comptime` refuses `extern` and native calls, because they have side effects

Two habits keep FFI code safe. Keep every `unsafe` block as small as you can, and write a thin KingCobra function around each C call so the rest of your program never sees raw pointers.

---

23. Work problems

Here is a practice set. Each problem tells you what to write, and the answer shown is **only the final value** your program should print. No solutions are given, so write the code yourself, run it, and compare. If your answer differs, use `print` to inspect your values step by step. Many answers use the same helper functions defined in the first problem of their group.

23.1 Numbers and operators (chapter 4)

Problem	Answer
`print(7 / 2)`	3.5
`print(8 / 2)`	4
`idiv(7, 2)`	3
`10 % 3`	1
`pow(2, 8)`	256
`5 ^ 3` (bitwise XOR)	6
`1 << 5`	32
`255 & 15`	15
`typeof(3 / 2)`	float
`typeof(4 / 2)`	int
`clamp(15, 0, 10)`	10
`max(3, 9)` and `min(3, 9)`	9 and 3
`abs(-9)`	9

23.2 Strings (chapter 5)

Problem	Answer
`len("kingcobra")`	9
`"hello"[1..4]`	ell
`"ab".repeat(3)`	ababab
Split `"a,b,c"` on commas and join with `-`	a-b-c

Problem	Answer
Count the vowels in `"KingCobra Language"` (ignore case)	7
`f"{1 + 2}-{"x".upper()}"`	3-X
Write `is_palindrome(s)`: `"racecar"` and `"cobra"`	true and false

23.3 Loops (chapter 7)

Problem	Answer
Sum of all numbers from 1 to 100 that are multiples of 3 or 5	2418
Count of primes below 50	15
Start `n = 0`, run `do { n += 4; } while (n < 10);`. Final `n`	12
Collatz steps for 27 (halve when even, else `3n + 1`, until 1)	111
Collatz steps for 6	8

23.4 Functions and recursion (chapter 8)

Problem	Answer
`fib(10)` with `fib(0) = 0`, `fib(1) = 1`	55
`fib(20)`	6765
`fact(5)`	120
`fact(10)`	3628800
`gcd(48, 18)` using Euclid's algorithm	6
`gcd(1071, 462)`	21
Sum of the digits of 12345	15
Binary representation of 10, as a string	1010
Binary representation of 255, as a string	11111111

23.5 Arrays, objects and match (chapters 6, 9, 10)

Problem	Answer
Sum of the squares of `[1, 2, 3, 4, 5]`, using `map` and `sum`	55
Keep elements greater than 2 in `[5, 3, 8, 1]`, using `filter`	[5, 3, 8]
`sorted([5, 3, 8, 1])`	[1, 3, 5, 8]
`var q = [0; 3]; q[2]++;` then print `q`	[0, 0, 1]
Sum the values of the object `{ a: 1, b: 2 }` with a `for (k, v in o)` loop	3
`match (7) { 1 => "low", 7 => "seven", _ => "other" }`	seven
A `Rectangle` struct with `w = 3`, `h = 4`: area and perimeter	12 and 14
The grade book from chapter 16: the average for Ada	91.6667

23.6 Preprocessor, C style and B style (chapters 19 to 21)

Problem	Answer
With `#define SQ(x) ((x) * (x))`, evaluate `SQ(1 + 2)`	9
`printf("%05d", 42)`	00042
`printf("%x", 255)`	ff
`printf("%b", 5)`	101
`strlen("hello")`	5
`toupper("ab")`	AB
Three calls to `next_id()` using a `static counter = 100` and `return counter++`	100, 101, 102
`qsort` the array `[9, 4, 7, 1]` with `x - y`, then `bsearch` for 7	[1, 4, 7, 9] and index 2
B style `gcd(1071, 462)` printed with `printn(..., 10)`	21
B style `printn(255, 16)`	ff

23.7 FFI (chapter 22)

Problem	Answer
`extern fn abs` from `libc.so.6`, call `abs(-42)`	42
`extern fn pow` from `libm.so.6`, call `pow(2.0, 10.0)`	1024
Allocate 16 bytes with C `malloc`, fill 15 with `0x41`, terminate, then `strlen`	15
The C file `mathutil.c` from 22.3, imported: `square(6)` and `cube(4)`	36 and 64

23.8 A challenge

Combine everything: write a program that defines a `Student` struct with an `average` method, stores three students in an array, sorts them by average with `qsort`, and prints each as `name: average` with `printf`. Make the average printing use `%.1f`. Then wrap the printing in a function that is only compiled when `-DVERBOSE` is passed. If your three students are Ada with scores 95, 88, 92, Bo with 72, 65, 80, and Cy with none, the average line for Ada should read `Ada: 91.7`.

---

24. Cheat sheet

```
// variables
let a = 1;   var b = 2;   const C = 3;

// decisions
if (x) { } else if (y) { } else { }
let r = match (n) { 0 => "zero", _ => "other" };
let t = cond ? a : b;     let v = maybe ?? "default";

// loops
while (c) { }     for (i in 0..10) { }     for (x in arr) { }     for (k, v
in obj) { }
break;  continue;  outer: for (...) { ... break outer; }

// functions
fn name(p: int, q: int = 1, ...rest: int) -> int { return p; }
let f = fn(x) -> int { return x; };

// data
let arr = [1, 2, 3];     let o = { k: 1 };
struct S { f: int }      let s = S { f: 1 };
impl S { fn m(self) -> int { return self.f; } }
enum E { A, B }          type T = int;

// errors and cleanup
try { throw error("x"); } catch (e) { print(e.message); }
defer cleanup();

// files
import "other.kc" as o;     import std.system;

// tools
kingcobra file.kc     kingcobra repl     kingcobra check file.kc
kingcobra test        kingcobra help topic
```

```
// preprocessor
#define NAME value      #define SQ(x) ((x) * (x))      #undef NAME
#ifdef X ... #else ... #endif      #if A > 1 && defined(B) ... #endif
#include "file.kc"      #pragma once      #define S(x) #x      #define C(a,
b) a##b

// C style
for (var i = 0, j = 9; i < j; i++, j--) { }     do { } while (c);
switch (x) { case 1, 2: ...; break; default: ... }     goto label;   label:
static n = 0;     [0; 8]     printf("%d %s\n", 1, "a");     fn main(argc,
argv) -> int { return 0; }

// B style
gcd(a, b) { auto t; ... return a; }     main() { auto v[10]; extrn putchar;
putchar(65); }
```

```
// FFI (calling C)
extern fn abs(x: int) -> int from "libc.so.6";
import "mylib.so" as lib;      lib.square(9);
unsafe { let p = malloc(16); free(p); }      // raw pointers need unsafe
```

---

25. Where to go next

Once this all feels familiar, the repository's `docs/` folder has the deeper references: `LANGUAGE.md` (complete reference), `GRAMMAR.md` (formal syntax), `POINTERS.md`, `MEMORY.md`, `C_AND_B.md` (the full reference for chapters 19 to 21, including `goto`, `printf` and the preprocessor) and `REPL.md`. The `examples/` folder, especially `language_tour.kc`, is a good place to read real code.

Happy coding!